

Binary Search

Imagine o seguinte conjunto de dados ordenado:

1 4 5 9 10 14 16 17 18 19 25 28 29 30

Se eu não soubesse em que posição está o número 28, o que eu teria que fazer para encontrar?

Você teria que percorrer elemento a elemento para encontrar.

Se nosso conjunto tivesse 1 milhão de elementos, teríamos que percorrer, no pior caso, 1 milhão de vezes para encontrar o número desejado.

1 4 5 9 10 14 16 17 18 19 25 28 29 30

Para resolver esse problema, vamos pensar em um joguinho de adivinhação:

- Imagine um programa em que um número de 1 a 100 é sorteado, e você deve tentar adivinhar esse número;
- A cada tentativa, o programa te diz se o número que você chutou é igual, menor ou maior que o número sorteado.

Sabendo que o número sorteado está entre 1 e 100, o que você chutaria primeiro?

Se chutarmos 50, e o programa diz que nosso número é menor que 50, é como se nosso número sorteado saísse de 1 a 100 para o novo intervalo de **1 a 49**.

Essa é a base da busca binária, sempre ‘chutamos’ um valor intermediário, e eliminamos metade do nosso intervalo a cada iteração.

Voltando para nosso conjunto, vamos tentar achar o número 19:

1 4 5 9 10 14 16 17 18 19 25 28 29 30

0 1 2 3 4 5 6 7 8 9 10 11 12 13

Temos 14 (de 0 a 13) índices, vamos aplicar a mesma ideia de ‘chutar’ o número misterioso:

- Vamos manter uma variável guardando o começo do intervalo (nesse caso 0)
- E outra variável guardando o fim do intervalo (nesse caso 13)

1 4 5 9 10 14 16 17 18 19 25 28 29 30

0



13



A iteração é da seguinte forma:

Pegamos o ponto médio do intervalo:

- Fazemos a soma das duas variáveis e dividimos por 2

$$\text{Meio} = (\text{Esquerda} + \text{Direita}) // 2$$

$$\text{Meio} = (0 + 13) // 2$$

$$\text{Meio} = 13 // 2$$

$$\text{Meio} = 6$$

1	4	5	9	10	14	16	17	18	19	25	28	29	30
0	1	2	3	4	5	6							

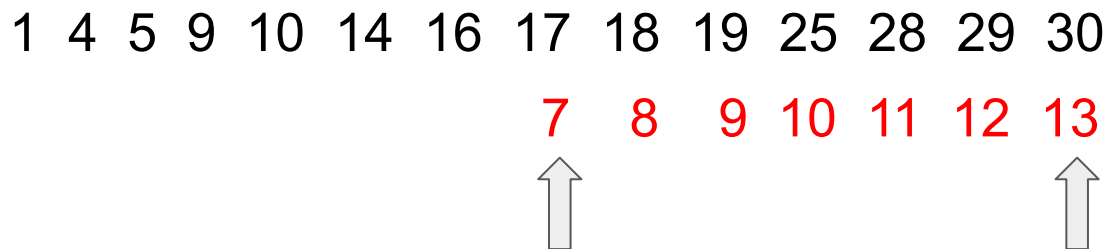


Agora comparamos o número que está no local da conta resultante.

Nesse caso, o índice **6** é o número 16.

E comparamos com o número 19:

- 16 é igual a 19? Não
- 16 é maior do que 19? Não
- 16 é menor do que 19? Sim



Se 16 é menor do que 16, eu sei que **não** preciso mais chutar valores dele para baixo.

Nosso novo intervalo virou de **7 a 13**.

Então minha variável Esquerda, agora é igual a 7.
E nossa variável Direita não mudou, continua 13.

Repetimos esse processo até encontrar o valor desejado:

1 4 5 9 10 14 16 17 18 19 25 28 29 30

7 8 9 10 11 12 13



Meio = (Esquerda + Direita) // 2

Meio = (7 + 13) // 2

Meio = 20 // 2

Meio = 10

1 4 5 9 10 14 16 17 18 19 25 28 29 30

10



1 4 5 9 10 14 16 17 18 19 25 28 29 30

10



Nesse caso, o índice **10** é o número 25.

E comparamos com o número 19:

- 25 é igual a 19? Não
- 25 é maior do que 19? Sim

Novo intervalo:

1 4 5 9 10 14 16 17 18 19 25 28 29 30

7

8

9



1 4 5 9 10 14 16 17 18 19 25 28 29 30

7 8 9



Meio = (Esquerda + Direita) // 2

Meio = (7 + 9) // 2

Meio = 16 // 2

Meio = 8

Nesse caso, o índice **8** é o número 18.

E comparamos com o número 19:

- 18 é igual a 19? Não
- 18 é maior do que 19? Não
- 18 é menor do que 19? Sim

Nosso novo intervalo:

1 4 5 9 10 14 16 17 18 19 25 28 29 30

9

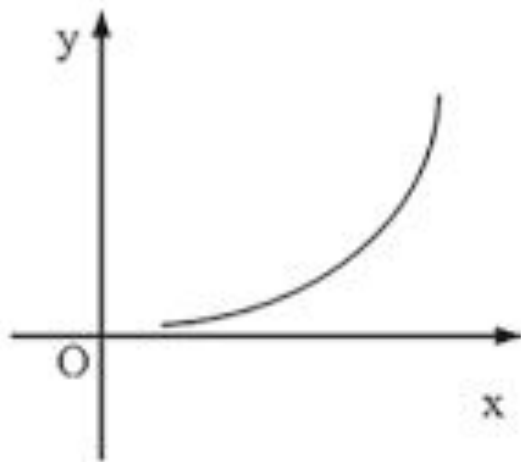


Achamos!

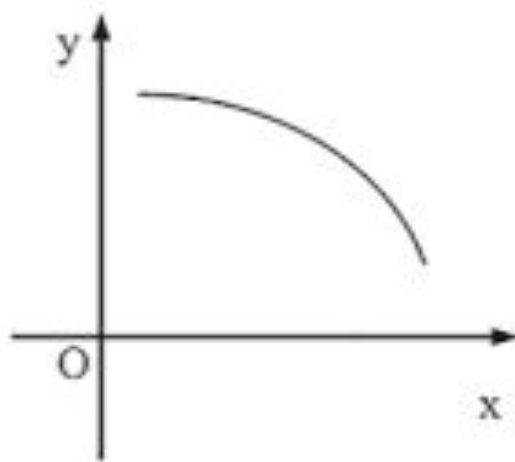
IMPORTANTE!!!!

A busca binária só funciona, se o conjunto de dados que estamos buscando for **monotônico**.

Ou seja, o conjunto está ordenado de forma crescente ou decrescente.
Em outras palavras, a função que representa o intervalo deve ser monotônica.



função crescente



função decrescente

```
dados = [1,4,5,9,10,14,16,17,18,19,25,28,29,30]

esquerda = 0
direita = 13
numero_adivinhar = 19
resposta = -1
v while esquerda <= direita:
    meio = (esquerda + direita) // 2

    chute = dados[meio]

v     if chute == numero_adivinhar:
        resposta = meio
        break

v     if chute > numero_adivinhar:
        direita = meio - 1
v     else:
        esquerda = meio + 1

print(resposta)
```

Por que essa técnica é importante?

Imagine um conjunto de dados que tenha 10^{18} números, ou seja, um conjunto de dados extremamente grande.

E eu preciso encontrar o índice de um valor que esteja lá para o fim desse conjunto. Se percorrermos elemento a elemento, podemos ter no pior caso 10^{18} iterações!

E de quantas operações eu preciso caso usasse busca binária?

Para descobrir, vamos lembrar que a cada iteração dividimos nosso conjunto no meio.

Podemos escrever o tamanho do nosso problema após cada iteração **k**:

- **Iteração 1:** Dividimos por 2 -> **$N // 2$**
- **Iteração 2:** Dividimos por 4 -> **$N // 4$**
- **Iteração 3:** Dividimos por 8 -> **$N // 8$**

Reescrevendo

- **Iteração 1:** Dividimos por 2 -> **$N // 2^1$**
- **Iteração 2:** Dividimos por 4 -> **$N // 2^2$**
- **Iteração 3:** Dividimos por 8 -> **$N // 2^3$**
- **Iteração k:** **$N // 2^k$**

No pior caso, dividimos o conjunto **k** vezes, até sobrar 1 elemento:

$$\frac{N}{2^k} = 1 \quad \longrightarrow \quad 2^k = N$$

Para resolver essa equação, podemos usar **log**:

$$k = \log_2(N)$$

Se nosso **N** for 10^{18} :

$$k = \log_2(10^{18})$$

$$k \approx 60$$

Ou seja, chegamos no mesmo resultado em apenas 60 iterações, ao invés de 10^{18}

Problema: Dado um número inteiro **N**, encontre o menor número inteiro positivo **X** tal que a equação abaixo seja verdadeira:

$$X^3 + X \geq N$$

Restrição: $1 \leq N \leq 10^{18}$

```
n = int(input())

esquerda = 0
direita = 10 ** 18

resposta = -1
✓ while esquerda <= direita:
    meio = (esquerda + direita) // 2

    valor = meio ** 3 + meio

    ✓ if valor >= n:
        resposta = meio
        direita = meio - 1
    ✓ else:
        esquerda = meio + 1

print(resposta)
```